

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Análisis de sistemas II

Requerimientos Ágiles, BDD, UX/UI y Desarrollo Inicial de API

Integrantes:

Anderson Cristofer Rodríguez Pivaral	9989-23-495
Pablo Jafet Morales Zacarías	9989-23-11401
Ricardo Emilio Guidos Pérez	9989-15-18163
Jaime Ricardo Garrido Giron	9989-23-21073

Guatemala 2 de octubre de 2026

1. INTRODUCCIÓN

El presente proyecto aborda la definición inicial de requerimientos mediante un enfoque ágil, con el propósito de organizar las necesidades del Sistema de Gestión de Ventas en incrementos pequeños, priorizados y verificables. La utilización de un Product Backlog permite ordenar las funcionalidades de acuerdo con el valor que aportan al negocio y establecer una base común para los componentes de BDD, UX/UI y desarrollo backend.

El ciclo de desarrollo incremental busca que el equipo pueda construir y validar primero las funcionalidades esenciales del sistema, recibir retroalimentación y posteriormente incorporar funcionalidades adicionales. De esta manera, el alcance puede evolucionar sin perder el control sobre las prioridades del producto.

El alcance inicial contempla el flujo básico de una venta: autenticación de usuarios autorizados, consulta de productos, validación de disponibilidad, registro de la venta y actualización del inventario. Las funcionalidades de consulta histórica, procesamiento de pagos y generación de comprobantes se organizan como incrementos posteriores según su prioridad.

2. REQUERIMIENTOS ÁGILES Y PRODUCT BACKLOG

2.1 Descripción de Épicas y Mínimo Producto Viable (MVP)

Épica 1 - Autenticación y acceso: Autenticación y acceso seguro.

Épica 2 - Pagos y transacciones: Procesamiento de cobros.

Épica 3 - Consulta y seguimiento de ventas: Historial por vendedor y cliente.

Épica 4 - Garantías y soporte: Validación de garantías.

Épica 5 - Indicadores y resumen de ventas: Indicadores consolidados por período.

El MVP incluye las seis funcionalidades representadas por los endpoints: autenticación/JWT, procesamiento de pagos, historial por vendedor, validación de garantía, historial de compras de cliente y resumen de ventas. La gestión de inventario queda fuera del alcance.

2.2 Criterios de Priorización y Estimación (Story Points)

Las historias se estiman mediante Story Points utilizando una escala Fibonacci de 1, 2, 3, 5, 8 y 13. La estimación representa de forma relativa la complejidad, esfuerzo e incertidumbre de cada historia. Para la priorización se utiliza MoSCoW.

Must Have corresponde a funcionalidades indispensables para el MVP; Should Have a funcionalidades importantes que pueden desarrollarse después del flujo mínimo; Could Have a funcionalidades deseables que pueden posponerse sin impedir la operación básica; y Won't Have a funcionalidades que no forman parte del incremento actual.

2.3 Tabla Consolidada de Historias de Usuario

ID	Historia	Endpoint	Método	Prioridad	SP
HU-01	Como usuario autorizado, quiero iniciar sesión con mis credenciales para acceder de forma segura a las funcionalidades del módulo de ventas.	POST /auth/login	POST	Must Have	3

HU-02	Como vendedor, quiero procesar el pago de una venta para completar de forma segura la transacción realizada por el cliente.	POST /payments/process	POST	Must Have	5
HU-03	Como responsable de recursos humanos, quiero consultar el historial de ventas de un vendedor para dar seguimiento a las ventas realizadas por cada vendedor.	GET /sales/rep/{vendedorId}	GET	Should Have	5
HU-04	Como personal de soporte, quiero validar la existencia y vigencia de la garantía asociada a una venta para determinar si el cliente puede recibir atención por garantía.	GET /sales/validate-warranty/{saleId}	GET	Should Have	5
HU-05	Como cliente, quiero consultar mi historial de compras para conocer las ventas y compras que he realizado anteriormente.	GET /sales/customer/{clientId}	GET	Should Have	5

HU-06	Como administrador o responsable de ventas, quiero consultar un resumen de los indicadores de ventas por período para analizar el comportamiento de las ventas realizadas.	GET /sales/summary	GET	Could Have	5
-------	--	--------------------	-----	------------	---

Justificación de la priorización

HU-01 y HU-02 son Must Have por corresponder al acceso seguro y procesamiento transaccional. HU-03, HU-04 y HU-05 son Should Have por aportar consulta, seguimiento y soporte. HU-06 es Could Have porque aporta análisis consolidado y puede incorporarse posteriormente sin impedir las operaciones principales.

Reglas de Negocio

RN-01: Los usuarios deben autenticarse para acceder a operaciones protegidas.

RN-02: Las operaciones protegidas utilizan Bearer JWT.

RN-03: Los pagos se procesan mediante el endpoint definido para cobros.

RN-04: El historial por vendedor utiliza el identificador del vendedor.

RN-05: La garantía se valida mediante el identificador de la venta.

RN-06: El historial de compras utiliza el identificador del cliente.

RN-07: El resumen presenta indicadores consolidados por período.

RN-08: La gestión de inventario no forma parte del alcance.

Vínculo al Tablero Ágil: [\[Abrir Tablero Ágil\]](#)

3. ESPECIFICACIONES BDD (CRITERIOS DE ACEPTACIÓN)

3.1 Cobertura de Escenarios Gherkin

Para garantizar la calidad del software y una clara alineación entre las definiciones de negocio del Product Owner y la implementación del equipo backend, se adoptó la metodología BDD (Behavior-Driven Development). Los criterios de aceptación de cada Historia de Usuario fueron especificados utilizando la sintaxis estandarizada Gherkin (Dado / Cuando / Entonces) en idioma español.

La estrategia de cobertura abarca:

- Escenarios Principales (Camino Feliz): Validaciones donde la ejecución ocurre de manera idónea, cumpliendo los objetivos del usuario y retornando respuestas HTTP 200 OK o 201 Created.
- Escenarios de Excepción y Error: Mapeo explícito de situaciones anómalas, incluyendo credenciales incorrectas, recursos no encontrados, errores de formato en parámetros y solicitudes no autorizadas sin token JWT (códigos 400, 401, 402, 404).

3.2 Matriz de Escenarios por Historia

ID	Historia de Usuario	Archivo .feature	Escenarios Felices	Escenarios de Error
HU-01	Autenticación y acceso con JWT	autenticacion.feature	1	2
HU-02	Procesamiento de pago de venta	gestion ventas.feature	1	2
HU-03	Consulta de historial por vendedor	gestion ventas.feature	1	1
HU-04	Validación de garantía de venta	gestion ventas.feature	1	1
HU-05	Consulta de historial por cliente	gestion ventas.feature	1	1
HU-06	Resumen de indicadores de ventas	gestion ventas.feature	1	1

Vínculo a Archivos .feature en el repositorio: [docs/agile/features/](#)

4. DISEÑO DE EXPERIENCIA DE USUARIO (UX/UI)

El prototipo del módulo de ventas representa el acceso al sistema, la consulta de información comercial y el procesamiento de pagos. Se diseñó en Figma con enfoque Responsive Web y una combinación de superficies Glassmorphism y organización Bento Grid. Los importes se presentan en quetzales guatemaltecos (Q) y el pago utiliza el código de moneda GTQ. Las interacciones son simulaciones de diseño: el prototipo no ejecuta solicitudes contra un backend ni acredita transacciones reales.

4.1 Mapa de Navegación del Sistema

El acceso comienza en la pantalla de inicio de sesión. La acción Ingresar conduce al dashboard, que concentra el resumen de ventas y los accesos a las consultas por vendedor, historial del cliente, validación de garantía y pago. La estructura permite localizar cada tarea desde un punto común y regresar al resumen al finalizar.

Mapa de navegación del prototipo

Inicio de sesión → Ingresar → Dashboard de ventas

Dashboard → Ventas por vendedor → Regresar al dashboard

Dashboard → Historial del cliente → Regresar al dashboard

Dashboard → Validación de garantía → Regresar al dashboard

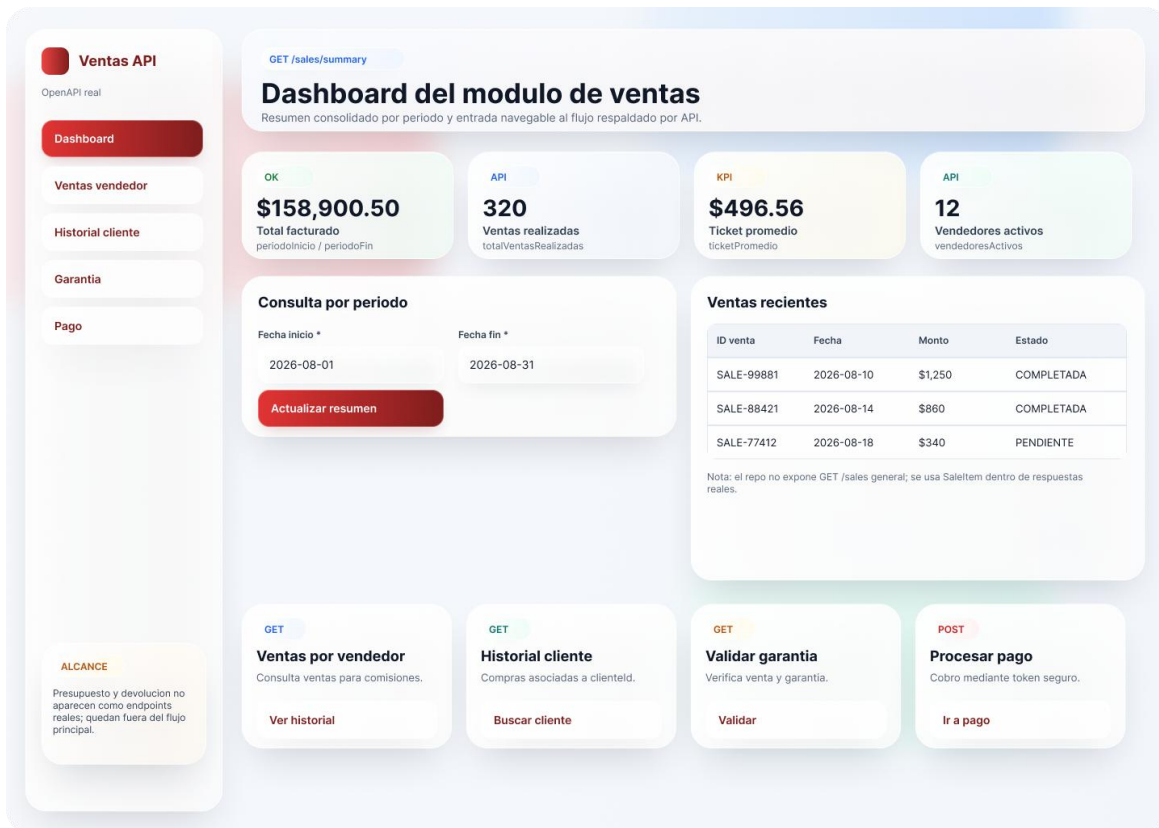
Dashboard → Procesar pago → Confirmar → Pago exitoso → Dashboard

Procesar pago → Simular error → Error de pago → Reintentar → Procesar pago

Error de pago → Cancelar → Dashboard

El resultado exitoso y el error de pago se representan mediante pantallas enlazadas; no corresponden a respuestas verificadas de una pasarela. En el inicio de sesión también se representa un mensaje de error. Estas vistas permiten revisar la comprensión de los mensajes y las alternativas de recuperación durante una demostración.

La composición de escritorio distribuye los bloques de información y acciones en columnas. Las adaptaciones móviles de dashboard y pago apilan los grupos en un orden de lectura vertical. Esta adaptación demuestra la intención responsive, pero los marcos de Figma no sustituyen una prueba de redistribución continua en un navegador. Las demás vistas requieren una comprobación específica en anchuras pequeñas.



4.2 Mapeo de Pantallas vs. Endpoints OpenAPI

El mapeo relaciona las acciones del prototipo con las operaciones documentadas en docs/api/openaip.yaml del repositorio AndersonRodPi/mi-proyecto-sistema2. Los métodos y rutas describen el contrato de integración previsto; su presencia en OpenAPI no demuestra que exista un servicio desplegado. La secuencia de lectura es Pantalla → Acción → Método y endpoint → Resultado esperado.

Inicio de sesión → Ingresar → POST /auth/login

Datos de entrada: Cuerpo JSON: email y password. La etiqueta del diseño indica Usuario / email, pero el contrato documenta email; no debe asumirse compatibilidad con un nombre de usuario independiente. Resultado esperado: Recepción de token, tipo de token y vigencia; en el prototipo se navega al dashboard.

Dashboard de ventas → Actualizar resumen → GET /sales/summary

Datos de entrada: Parámetros de consulta: startDate y endDate. Resultado esperado: Resumen de ventas del período seleccionado para alimentar los indicadores.

Ventas por vendedor → Buscar ventas → GET /sales/rep/{vendedorId}

Datos de entrada: vendedorId en la ruta; startDate y endDate como filtros opcionales. Resultado esperado: Información de ventas asociada al vendedor.

Historial del cliente → Buscar cliente → GET /sales/customer/{clienteId}

Datos de entrada: clienteId en la ruta. Resultado esperado: Historial de compras asociado al cliente consultado.

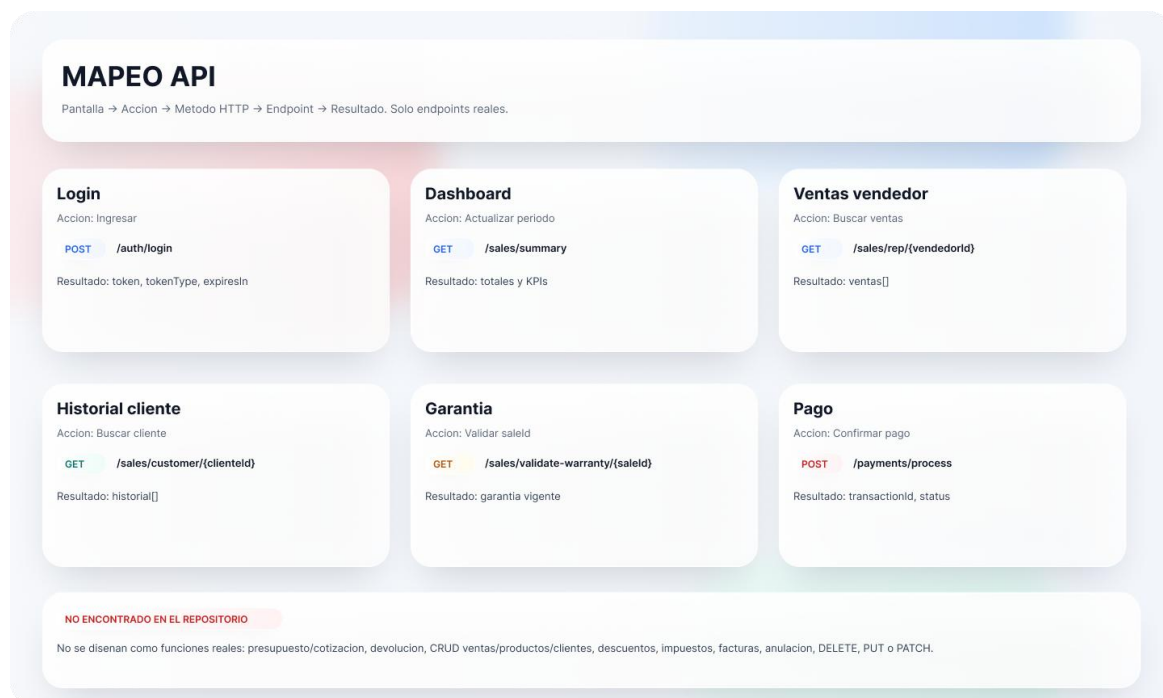
Validación de garantía → Validar venta → GET /sales/validate-warranty/{saleId}

Datos de entrada: saleId en la ruta. Resultado esperado: Resultado de la validación de garantía de la venta.

Procesar pago → Confirmar pago → POST /payments/process

Datos de entrada: Cuerpo JSON: amount, currency y cardToken. Para Guatemala se representa currency = GTQ. Resultado esperado: Resultado de transacción; se representa mediante vistas de éxito o error.

El pago se modela con un token de tarjeta y no con el envío de número de tarjeta o CVV al servicio propio. Las rutas anteriores son las operaciones representadas en este alcance. Presupuestos, cotizaciones y devoluciones no se presentan como operaciones respaldadas por el contrato revisado. La validación de garantía es una consulta y no equivale a registrar una devolución.



4.3 Reporte de Evaluación de Usabilidad y Accesibilidad (WCAG 2.1)

La evaluación aborda legibilidad, contraste, campos de formulario, navegación y áreas de interacción. Se basa en la configuración del plugin que genera el diseño y en la documentación local del prototipo. Los resultados que dependen del fondo compuesto, de la interacción real o de tecnologías de asistencia requieren verificación adicional. No se declara conformidad integral con WCAG 2.1.

Contraste de texto y controles (1.4.3 y 1.4.11)

El diseño configura texto principal #111827 y texto secundario #596579 sobre superficies claras. Los paneles incorporan transparencia y desenfoco, por lo que la relación de contraste debe medirse sobre el fondo visible final. Para texto normal se requiere al menos 4.5:1; para texto grande, 3:1. Los límites o indicadores visuales necesarios para identificar controles deben alcanzar 3:1 respecto al color adyacente. Los botones primarios utilizan un degradado

rojo con texto blanco y los campos normales un borde blanco: ambos requieren revisión específica. Resultado: contraste final pendiente de medición; una selección de colores por sí sola no acredita cumplimiento.

Tipografía, ampliación y redistribución (1.4.4 y 1.4.10)

La interfaz utiliza Inter, con texto habitual de 14 px y etiquetas de campos de 12 px en el generador. Se recomienda aumentar las etiquetas pequeñas cuando afecten la lectura. WCAG 2.1 no impone una tipografía o tamaño base único: exige que el contenido conserve su función al ampliar el texto al 200 % y que se redistribuya, con las excepciones del criterio, a una anchura equivalente a 320 píxeles CSS. Las vistas móviles creadas tienen 390 px de ancho. Resultado: adaptación visual parcial; quedan pendientes las pruebas a 320 px y de ampliación. La tipografía de este documento es Times New Roman de 12 puntos y es independiente de la tipografía de la interfaz.

Áreas de toque (2.5.5, nivel AAA)

El generador establece 46 px de altura para botones y campos. Esta dimensión supera la referencia de 44 px de altura del criterio 2.5.5, pero el cumplimiento exige comprobar también anchura, superficie realmente interactiva y excepciones aplicables. Es un criterio de nivel AAA en WCAG 2.1; no debe presentarse como requisito AA. Resultado: altura favorable en el diseño, con comprobación pendiente de todos los objetivos en la presentación final.

Etiquetas y mensajes de error (3.3.1 y 3.3.2)

Los formularios incluyen etiquetas persistentes y asterisco en los campos marcados como obligatorios. Se representan estados de error en inicio de sesión y pago. Debe explicarse el significado del asterisco y cada mensaje debe identificar el problema y orientar su corrección, sin depender solo de un cambio de color. Resultado: recursos visuales presentes; quedan por comprobar su asociación con cada campo y su anuncio mediante tecnologías de asistencia en la implementación.

Foco y navegación (2.1.1, 2.4.3, 2.4.7 y 3.2.3)

El estado de foco de los campos se representa con borde azul y un contorno adicional. La navegación enlazada permite recorrer las tareas principales y regresar al dashboard; los recorridos de éxito, error, reintento y cancelación facilitan la recuperación. La existencia de un estado visual de foco no demuestra navegación por teclado. Resultado: representación visual disponible; deben comprobarse tabulación, orden del foco, activación por teclado, ausencia de bloqueos y consistencia de los accesos en la aplicación implementada.

La revisión identifica una base visual útil para demostrar el módulo, junto con verificaciones pendientes de contraste, ampliación, redistribución y uso por teclado. El reporte complementario se referencia en docs/ui/reporte_usabilidad.pdf. Las evidencias de esta sección corresponden al diseño y su configuración; no se incluyen resultados de ejecución de API o de Postman.

Vínculo al Prototipo Interactivo en Figma: [Abrir Prototipo Figma](#)

5. ARQUITECTURA DEL CASCARÓN BACKEND (BOILERPLATE)

5.1 Explicación de la Estructura de Capas Elegida

En alineación con la Arquitectura de Microservicios definida para el sistema, se implementó el Microservicio de Ventas utilizando Java con Spring Boot. Internamente, el código fuente de este microservicio sigue un patrón de Arquitectura en Capas, lo que permite separar claramente las responsabilidades de recepción de peticiones, lógica de negocio y persistencia de datos.

La estructura interna del microservicio dentro de `src/main/java/com/umg/apiventas/` se organiza en las siguientes capas:

- **Capa de Controladores (controllers):** Expone los endpoints HTTP/REST del microservicio. Recibe las solicitudes externas, valida los parámetros de entrada y devuelve las respuestas JSON con los códigos de estado HTTP adecuados.
- **Capa de Transferencia de Datos (dto):** Objetos de transferencia (DTO) que definen el contrato del API REST, aislando las entidades internas de la base de datos de los esquemas expuestos al exterior.
- **Capa de Lógica de Negocio (services):** Contiene la lógica del microservicio de ventas (procesamiento de transacciones de pago, consultas de rendimiento por vendedor y autenticación).
- **Capa de Persistencia (repositories):** Abstracción de acceso a datos mediante Spring Data JPA, encargada de ejecutar las operaciones CRUD contra el esquema relacional.
- **Capa de Dominio / Modelos (models):** Mapeo de entidades JPA que representan la estructura tabular del microservicio en MySQL.

5.2 Configuración de Conexión a Base de Datos

La conexión con el motor relacional MySQL se gestiona mediante el archivo de propiedades del framework (`src/main/resources/application.properties`).

Se configuró el conector `com.mysql.cj.jdbc.Driver` utilizando el pool de conexiones HikariCP (incluido en Spring Boot). Además, la estrategia de DDL se estableció en `validate` para garantizar que Spring valide las entidades contra la estructura SQL existente sin modificar las tablas ni alterar los datos.

Fragmento de configuración utilizada (`application.properties`):

```
spring.application.name=api-ventas
```

```
# Conexión JDBC a MySQL
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/modulo_ventas?useSSL=false&serverTimezone=UTC
```

```
spring.datasource.username=${DB_USER:root}
```

```
spring.datasource.password=${DB_PASS:password}
```

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

```
# Configuración de JPA / Hibernate
```

```
spring.jpa.hibernate.ddl-auto=validate
```

```
spring.jpa.show-sql=true
```

```
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
```

Nota: Para dar cumplimiento a las buenas prácticas de seguridad exigidas en el proyecto, se incluyó el archivo `.env.example` en la raíz del repositorio indicando la estructura de las variables de entorno necesarias para la conexión.

5.3 Diagrama de Secuencia del Endpoint Principal

Vínculo al Diagrama de Secuencia: docs/architecture/secuencia_endpoint.png

6. IMPLEMENTACIÓN Y PRUEBA DE ENDPOINTS INICIALES

6.1 Detalle del Endpoint de Lectura (GET) Refactorizado

El endpoint de lectura implementado permite consultar el historial de transacciones asociadas a un vendedor específico a partir de su identificador único.

- **URL del Endpoint:** `/api/v1/sales/rep/{vendedorId}`
- **Método HTTP:** GET
- **Parámetros de Ruta (Path Variable):** `vendedorId` (String) - Código identificador del vendedor (ejemplo: VEND-1024).
- **Código de Respuesta Exitoso:** 200 OK
- **Estructura DTO de Respuesta (VendedorVentasResponse):**

```
[
  {
    "ventaId": "SALE-99881",
    "clienteNombre": "Juan Pérez",
    "fecha": "2026-08-10T08:30:00",
    "total": 1250.00,
    "estado": "COMPLETADA"
  }
]
```

6.2 Detalle del Endpoint de Escritura (POST) Refactorizado

Se implementó el endpoint de procesamiento de pagos como operación de escritura principal. Este servicio recibe la solicitud de cobro, genera el registro de la transacción en la tabla `pagos` y actualiza automáticamente el estado de la venta asociada a `COMPLETADA`.

- **URL del Endpoint:** /api/v1/payments/process
- **Método HTTP:** POST
- **Headers:** Content-Type: application/json
- **Código de Respuesta Exitoso:** 201 Created
- **Cuerpo de la Solicitud (PaymentRequest):**

```
{
  "ventaId": "SALE-99881",
  "metodoPago": "TARJETA_CREDITO",
  "monto": 1250.00
}
```

- **Cuerpo de la respuesta (PaymentResponse):**

```
{
  "exito": true,
  "mensaje": "Pago procesado exitosamente",
  "transaccionPasarela": "TX-545A5DDB"
}
```

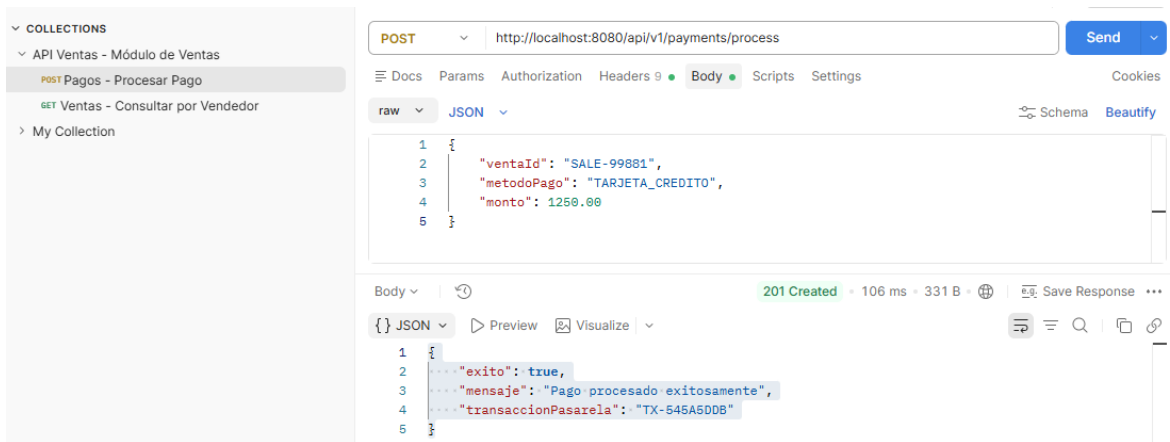
6.3 Evidencias de Ejecución (Postman/Insomnia)

Todas las pruebas funcionales de lectura y escritura fueron ejecutadas y verificadas de manera local contra la base de datos MySQL modulo_ventas.

Evidencias de ejecución de Endpoints de lectura (GET) y escritura (POST) en Postman:

The screenshot shows the Postman interface. On the left, the 'COLLECTIONS' panel is visible, showing a collection named 'API Ventas - Módulo de Ventas' with a sub-collection 'GET Ventas - Consultar por Vendedor'. The main area displays a GET request to the endpoint 'http://localhost:8080/api/v1/sales/rep/VEND-1024'. The 'Params' tab is active, showing a table with columns 'Key', 'Value', and 'Description'. The 'Body' tab is also visible, showing the response body in JSON format. The response status is '200 OK' with a response time of '1.99 s' and a size of '365 B'. The JSON response is as follows:

```
[
  {
    "ventaId": "SALE-99881",
    "clienteNombre": "Juan Pérez",
    "fecha": "2026-08-10T08:38:00",
    "total": 1250.00,
    "estado": "COMPLETADA"
  }
]
```



La colección estructurada de peticiones HTTP con sus parámetros y datos de prueba preconfigurados se encuentra disponible en la carpeta de pruebas del proyecto:

Vínculo a la Colección de Pruebas: [test-api/coleccion_postman.json](#)

7. MATRIZ DE TRAZABILIDAD ÁGIL

La siguiente matriz asegura la cobertura integral del ciclo de vida del desarrollo, vinculando cada requerimiento desde la Historia de Usuario hasta la implementación en código y pruebas automatizadas:

Requisito / HU	Archivo BDD (.feature)	Pantalla Figma	Endpoint API	Componente Backend
HU-01	autenticacion.feature	Login / Acceso	POST /api/v1/auth/login	AuthController.java
HU-02	gestion_ventas.feature	Pasarela / Cobro	POST /api/v1/payments/process	PaymentController.java
HU-03	gestion_ventas.feature	Dashboard Vendedor	GET /api/v1/sales/rep/{vendedorId}	SalesController.java / SalesService.java
HU-04	gestion_ventas.feature	Consulta Garantía	GET /api/v1/sales/validate-warranty/{saleId}	WarrantyController.java / WarrantyService.java
HU-05	gestion_ventas.feature	Mis Compras	GET /api/v1/sales/customer/{clientId}	CustomerController.java / CustomerService.java
HU-06	gestion_ventas.feature	Panel Indicadores	GET /api/v1/sales/summary	AnalyticsController.java / AnalyticsService.java

8. CONCLUSIONES Y LECCIONES APRENDIDAS

Alineación del ciclo ágil y especificación BDD: La adopción del marco ágil junto con la redacción de escenarios BDD en formato Gherkin permitió transformar requerimientos ambiguos en criterios de aceptación claros y verificables. Esto facilitó que los equipos de diseño y desarrollo compartieran una visión unificada del comportamiento del módulo de ventas ante flujos exitosos y de error desde las fases tempranas del proyecto.

Efectividad del prototipado UX/UI y evaluación de usabilidad: El diseño interactivo en Figma alineado a un mapa de navegación estructurado redujo la incertidumbre visual y funcional. La aplicación de criterios WCAG 2.1 garantizó la accesibilidad del sistema en áreas clave como el contraste de interfaces y la disposición de elementos, asegurando que la experiencia de usuario cumpla con estándares mínimos de calidad antes de la codificación.

Desacoplamiento y estandarización del Backend mediante OpenAPI: La definición previa del contrato de interfaz (OpenAPI/Swagger) actuó como el punto de acuerdo central entre la capa de presentación y la lógica de negocio en Spring Boot. Esto permitió estructurar endpoints estandarizados con prefijos explícitos (`/api/v1/...`), garantizando la correcta integración de servicios clave como la autenticación mediante JWT y el procesamiento transaccional de pagos.

Importancia de la Matriz de Trazabilidad: La construcción de la matriz de trazabilidad demostró ser un mecanismo indispensable para auditar la cobertura total del proyecto. Al vincular cada Historia de Usuario con su respectivo archivo `.feature`, vista en Figma, endpoint expuesto y componente Java (`Controller/Service`), se aseguró la consistencia técnica del sistema y se identificaron brechas de desarrollo de forma oportuna.

9. REFERENCIAS

1. **Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... & Thomas, D.** (2001). *Manifesto for Agile Software Development*. Agile Alliance.
2. **Cohn, M.** (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional.
3. **OpenAPI Initiative.** (2021). *OpenAPI Specification v3.1.0*. OAI Development Group. <https://spec.openapis.org/oai/v3.1.0>
4. **Smart, J. F.** (2014). *BDD in Action: Behavior-driven development for the whole software lifecycle*. Manning Publications.
5. **Walls, C.** (2019). *Spring in Action* (5th ed.). Manning Publications.
6. **World Wide Web Consortium (W3C).** (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C Recommendation. <https://www.w3.org/TR/WCAG21/>
7. **Wynne, M., & Hellesøy, A.** (2017). *The Cucumber Book: Behaviour-Driven Development for Testers and Developers* (2nd ed.). Pragmatic Bookshelf.